

经典终端（Terminal）交互 Bug 图谱与 E2E 测试建议（Deep Research）

日期：2026-02-12

范围：交互式 CLI/REPL（以 Windows 11 + PowerShell 7.x/Windows Terminal/ConPTY 为重点，同时覆盖 POSIX PTY 通用问题）

Executive Summary

交互式 CLI 的“经典坑”高度集中在三条链路：**键盘输入编码/按键语义（BS vs DEL、控制台模式）、输出重绘与流式输出竞争（prompt repaint/光标/回车）、以及 Unicode 显示宽度与组合字符（CJK/emoji/combining）**。这些问题跨语言/跨框架反复出现，且非常适合用端到端（E2E）测试固化回归，因为单元测试很难覆盖真实终端的行为差异。[1][2][3]

对 `openagentic_cli` 来说，最具价值的策略是：把 E2E 断言尽量锚到**可观测证据链**（如 session 的 `events.jsonl`），同时用“真 TTY”（Windows: ConPTY；POSIX: pty）覆盖输入序列、粘贴模式与重绘路径，从而在真实网络调用之外，再补齐“终端语义回归”。[4]

Key Findings

- **Backspace 是经典兼容性坑（BS=0x08 vs DEL=0x7f）**：不同终端/配置会让“退格键”发送不同控制码，导致程序端看到的字节不同，进而出现“删不掉 / 一下删一大片（删词）”等现象。[5][6]
- **Windows 控制台模式会改变按键处理方式**：开启 `ENABLE_VIRTUAL_TERMINAL_INPUT` 以接收 VT 序列（如 bracketed paste）是常见需求，但若同时保留行编辑/回显相关模式，按键可能被控制台“预处理”，造成语义漂移（例如 DEL 被解释成不同删除行为）。[1][2]
- **宽字符/组合字符导致“删不干净/残影/光标错位”**：CJK 全角字符通常占 2 列，组合附加符号可能占 0 列；若按“1 字符=1 列”擦除，极易在终端显示上留下残影或删不干净。[3][7]
- **输入与流式输出竞争会产生“吃字/整行消失（视觉/功能）”**：REPL 在用户输入时如果同时不断向 stdout 输出（尤其是 streaming delta），终端重绘会覆盖用户的行编辑区域；若同步点依赖 prompt 文案会更脆弱。[8]
- **编码与 code page 问题会在 Windows 上反复出现**：Windows console 存在输入/输出 code page；程序若假设 UTF-8，在非 UTF-8 code page 下会把中文解码成乱码，进一步影响“字符宽度/退格擦除/日志”链路。[9]

Detailed Analysis

1) Backspace/DEL：为什么“经典”且难缠

在 VT/终端世界里，“退格键”可能表现为：

- **BS (0x08)**：Backspace 控制字符
- **DEL (0x7f)**：历史上也常被当作 Backspace（尤其是 xterm/兼容终端在某些配置下）

因此同一个“按下 backspace”的用户动作，程序端可能看到不同字节序列，若实现只处理其中一种，就会出现“删不动”或“删错”的体验。[5][6]

更麻烦的是：即便你处理了 BS/DEL，两端还可能出现“系统行编辑”插手（见下一节），把按键从“删除一个字符”提升为“删除一个词/整段”。这类问题在交互式 CLI、Readline/line editor、以及 Windows console 体系里都非常常见，所以它非常“经典”。[1][2]

2) Windows Console Mode：VT input vs 行编辑/回显

在 Windows 上，想要支持 bracketed paste（`ESC[200~ ... ESC[201~`）等 VT 输入序列，需要开启 `ENABLE_VIRTUAL_TERMINAL_INPUT`。[2]

但 Windows 控制台还有一系列与“行输入/回显/预处理”相关的模式。微软文档明确说明：当启用 `ENABLE_LINE_INPUT` 时，系统会在应用收到输入前进行行缓冲；当启用 `ENABLE_ECHO_INPUT` 时，系统会回显用户输入；当启用 `ENABLE_PROCESSED_INPUT` 时，系统会处理某些按键（如 Ctrl+C 等）。这意味着应用看到的输入并不一定是“原始按键字节流”。[1][2]

对交互式 CLI 来说，一个常见的工程取舍是：

- 要么依赖系统行编辑（省事，但语义不可控，且与 VT input 组合时容易出“删词/吞键”等怪相）
- 要么切到更“raw”的读取方式，自己实现最小行编辑（成本更高，但语义稳定、可测试、可回归）

3) Bracketed Paste：功能与安全双重价值

Bracketed paste 的设计目标之一，是让程序区分“键入”与“粘贴”，从而避免粘贴内容触发危险行为（例如：把多行粘贴当成逐行执行）。xterm 的文档描述了 bracketed paste 的启用方式与开始/结束标记（`ESC[200~`、`ESC[201~`），这也是现代终端与 TUI/REPL 常用的兼容机制。[4]

对 `openagent_cli` 这种“既有 REPL 命令（`/help`）又能把文本发给模型”的程序来说，***“粘贴的 `/help` 不能被当作命令执行”***就是典型的 bracketed paste 价值点：既减少误触发，也能降低 prompt 注入式的“本地命令误执行”风险。

4) Unicode: CJK、emoji、combining 与“删不干净”

终端显示不是“字符数”，而是“列宽”。在 POSIX 世界里，`wcwidth()` / `wcswidth()` 类函数就是为“字符占几列”服务的；很多 CJK 字符宽度是 2 列，而某些 combining marks 宽度为 0。[7]

如果 CLI 自己做了输入回显（例如 Windows raw 模式下为了规避系统行编辑），那么退格擦除必须按列宽来做，而不能简单 `\b \b` 一次就结束，否则会出现：

- 中文删不干净（残影）
- 光标位置错位（后续输入覆盖到奇怪的位置）
- emoji/变体选择符导致的“删半个图形/删完还有残留”

目前工程实践里通常把它分成两档：

- **最小可用**：按 East Asian Width 近似处理宽字符（W/F=2，其它=1，combining=0）
- **更完整**：按 grapheme cluster（用户感知字符）回删，需要 UAX #29 级别的分词/聚类（通常要引入额外库）

5) Windows Code Page：乱码与隐性连锁

Windows console 的输入/输出编码由 code page 决定。微软提供了 `GetConsoleCP`（输入）与 `GetConsoleOutputCP`（输出）等 API，这意味着应用若一律按 UTF-8 解码，在非 UTF-8 code page 下会把多字节序列解成乱码。[9]

而“乱码”不只是显示问题：它会进一步影响“字符宽度估算”“退格擦除列宽”“日志/落盘”“E2E 断言”等链路，从而把一个简单问题放大成一串看似无关的 flaky 行为。

Areas of Consensus

- 交互式 CLI 的 bug 很多不是业务逻辑，而是终端语义：按键码、控制台模式、宽字符、重绘与输出竞争。[1][3][5]
- “真终端”E2E（ConPTY/pty）比 pipes 更能捕捉真实回归，特别是输入序列与重绘相关的问题。[4]
- Windows code page/编码问题需要被显式纳入工程策略（至少在 E2E 与诊断上）。[9]

Areas of Debate

- **最小实现 vs 完整 line editor**：完全实现 Readline/PSReadLine 级编辑会很复杂；但仅做 minimal editing 又可能缺失用户期待（左右方向键、Home/End、Ctrl+Backspace 等）。

- **按 code point 回删 vs 按 grapheme cluster 回删**：后者用户体验最好，但实现与依赖成本更高（且跨平台一致性也更难）。
- **prompt 重绘策略**：为了更美观的输入区域而做 `\r` /ANSI 重绘，常会显著增加“输出竞争导致的视觉问题”；是否牺牲美观换稳定，是产品取舍。

Candidate E2E Tests（面向 openagentic_cli）

以下是“经典终端坑 → 可回归 E2E”的映射建议（按优先级）：

1. 退格与删除键

- Backspace as DEL(0x7f)：应只删除 1 字符，不应删词
- Backspace as BS(0x08)：同上
- CJK 宽字符退格：不残影、不乱码

2. 粘贴语义

- bracketed paste 中包含 `/help`：不得触发 REPL help 分支
- `/paste` 模式：多行合并为一个 turn，且包含空行与以 `/` 开头行时不被当命令

3. 输出竞争与 typeahead

- 响应中提前输入（含不按回车/按回车两种）：不得丢行、不得合并 turn

4. 编码/代码页

- 在非 UTF-8 code page 下输入中文：应正确落盘到 `events.jsonl`

5. 特殊按键与控制序列（可选增强）

- 箭头/Home/End：至少不应注入控制字符到 prompt 文本
- Ctrl+C：应中断当前请求或提示（不崩溃、不破坏 session）

Gaps and Further Research

- 把“宽字符 + combining + emoji + variation selector”的 grapheme cluster 级退格体验做成可选增强（可能需要额外依赖）。
- Windows Terminal / ConHost / VSCode Terminal / ConPTY 在键盘协议上的细微差异，需要通过更广泛的样本与 issue 调研来完善“兼容矩阵”。
- 如果要把 `conpty-expect` 演进为通用框架，可补：日志记录（输入/输出原始字节）、屏幕快照、可复现 seed 与 stress 模式。

Sources

- [1] Microsoft Docs — Console Modes (`GetConsoleMode`) (authoritative; explains input flags like `LINE_INPUT/ECHO_INPUT/PROCESSED_INPUT`). <https://learn.microsoft.com/en-us/windows/console/getconsolemode>
- [2] Microsoft Docs — Console Virtual Terminal Sequences (authoritative; describes VT input and related Windows console behavior). <https://learn.microsoft.com/en-us/windows/console/console-virtual-terminal-sequences>
- [3] Thomas E. Dickey — xterm control sequences / terminal behavior docs (authoritative; background on terminal features used by REPL/TUI). <https://invisible-island.net/xterm/ctlseqs/ctlseqs.html>
- [4] Thomas E. Dickey — xterm “Bracketed Paste Mode” (authoritative; defines `ESC[200/ESC[201` markers). <https://invisible-island.net/xterm/ctlseqs/ctlseqs.html#h3-Bracketed-Paste-Mode>
- [5] Stack Overflow — “Terminal emulator backspace key” (secondary; summarizes BS vs DEL variance across terminals; good practical overview). <https://stackoverflow.com/questions/14876612/terminal-emulator-backspace-key>
- [6] ncurses/terminfo manual — `key_backspace` capability (authoritative; terminfo mapping for backspace key sequences). <https://invisible-island.net/ncurses/man/terminfo.5.html>
- [7] Linux man-pages — `wcwidth(3)` / `wcswidth(3)` (authoritative; explains terminal column width calculation for wide/combining chars). <https://man7.org/linux/man-pages/man3/wcwidth.3.html>
- [8] Microsoft/windows-terminal GitHub repository (high-signal engineering source; contains ConPTY/Terminal input-output behavior discussions and issues). <https://github.com/microsoft/terminal>
- [9] Microsoft Docs — Console code pages (`GetConsoleCP` , `GetConsoleOutputCP`) (authoritative; explains Windows console encoding mechanics). <https://learn.microsoft.com/en-us/windows/console/getconsolecp>